PALADIN
BLOCKCHAIN SECURITY

Final Report

Smart Contract Security Assessment

For EtherFi

DATE

27 July 2026

paladinsec.co

x.com/0xPaladinSec

Table of Contents

1. Disclaimer

Paladin Blockchain Security ("Paladin") has conducted an independent audit to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the codes that were provided for the scope of this audit. This audit report does not constitute agreement, acceptance or advocation for the Project that was audited, and users relying on this audit report should not consider this as having any merit for financial advice in any shape, form or nature. The contracts audited do not account for any economic developments that may be pursued by the Project in question, and that the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are completely free of exploits, bugs, vulnerabilities or deprecation of technologies. Further, this audit report shall not be disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Paladin.

All information provided in this report does not constitute financial or investment advice, nor should it be used to signal that any persons reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report. Information is provided 'as is', and Paladin is under no covenant to the completeness, accuracy or solidity of the contracts audited. In no event will Paladin or its partners, employees, agents or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this audit report.

Cryptocurrencies and any technologies by extension directly or indirectly related to cryptocurrencies are highly volatile and speculative by nature. All reasonable due diligence and safeguards may yet be insufficient, and users should exercise considerable caution when participating in any shape or form in this nascent industry.

The audit report has made all reasonable attempts to provide clear and articulate recommendations to the Project team with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts provided. It is the sole responsibility of the Project team to sufficiently test and perform checks, ensuring that the contracts are functioning as intended, specifically that the functions therein contained within said contracts have the desired intended effects, functionalities and outcomes of the Project team.

Paladin retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. Paladin is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. Paladin is furthermore allowed to claim bug bounties from third-parties while doing so.

2. Overview

This report has been prepared for EtherFi's contracts on the Optimism network. Paladin provides a user-centred examination of the smart contracts to look for vulnerabilities, logic errors or other issues from both an internal and external perspective.

2.1 Summary

Project Name	EtherFi
Project URL	https://www.ether.fi/
Platform	Optimism
Language	Solidity
Initial Commit	https://github.com/etherfi-protocol/cash-v3/tree/3c932071c6d9930e71d46caad023e2700487e15f
Resolution #1	https://github.com/etherfi-protocol/cash-v3/tree/ac7092621579e5bbe630104dbd22e1d9d9f78470

2.2 Contracts Assessed

CONTRACT	CONTRACT ADDRESS	LIVE CODE MATCH
BaseAaveV4PriceFeed	–	Pending
StablePriceLib	–	Pending
ChainlinkPriceFeed	–	Pending
PythPriceFeed	–	Pending
VedaAccountantPriceFeed	–	Pending
OracleSinkPriceFeed	–	Pending
IAaveV4PriceFeed	–	Pending
IVedaAccountant	–	Pending
IOracleSink	–	Pending

2.3 Findings Summary

SEVERITY	FOUND	RESOLVED	PARTIALLY RESOLVED	ACKNOWLEDGED
GOVERNANCE	0	-	-	-
HIGH	0	-	-	-
MEDIUM	0	-	-	-
LOW	0	-	-	-
INFORMATIONAL	5	2	1	2
Total	**5**	**2**	**1**	**2**

ISSUE CLASSIFICATION

SEVERITY	DESCRIPTION
GOVERNANCE	Issues under this category are where the governance or owners of the protocol have certain privileges that users need to be aware of, some of which can result in the loss of user funds if the governance's private keys are lost or if they turn malicious, for example.
HIGH	Exploits, vulnerabilities or errors that will certainly or probabilistically lead towards loss of funds, control, or impairment of the contract and its functions. Issues under this classification are recommended to be fixed with utmost urgency.
MEDIUM	Bugs or issues with that may be subject to exploit, though their impact is somewhat limited. Issues under this classification are recommended to be fixed as soon as possible.
LOW	Effects are minimal in isolation and do not pose a significant danger to the project or its users. Issues under this classification are recommended to be fixed nonetheless.
INFORMATIONAL	Consistency, syntax or style best practices. Generally pose a negligible level of risk, if any.

BaseAaveV4PriceFeed

ID	SEVERITY	SUMMARY	STATUS
01	INFO	Gas observations	PARTIAL

ChainlinkPriceFeed

ID	SEVERITY	SUMMARY	STATUS
02	INFO	Redundant `SafeCast` after sign check	ACKNOWLEDGED

PythPriceFeed

ID	SEVERITY	SUMMARY	STATUS
03	INFO	Naming and code-quality observations	RESOLVED

VedaAccountantPriceFeed

ID	SEVERITY	SUMMARY	STATUS
04	INFO	Redundant second accountant call	RESOLVED

OracleSinkPriceFeed

ID	SEVERITY	SUMMARY	STATUS
05	INFO	Redundant `SafeCast` after sign check	ACKNOWLEDGED

3. Findings

3.1 BaseAaveV4PriceFeed

`BaseAaveV4PriceFeed` is the abstract base shared logic by every concrete Aave v4 receipt-token price feeds. It owns the two-mode USD composition in `_composeUsd`: a source rate is either scaled straight to the reported feed decimals when the source is already USD-quoted, or multiplied by an underlying asset's USD price – read from another IAaveV4PriceFeed, which enforces its own staleness – when the source is a rate quoted in that underlying asset.

The base also owns the stable-price snap, the immutable `decimals()` / `description()` metadata, the rejection of a zero or non-positive composed price, and the shared error set.

Derived feeds supply only the raw rate and its decimals from their own source; this base performs the compose and reports a `feedDecimals`-scaled USD price through the `IAaveV4PriceFeed` interface.

Privileged Functions

None

Issues & Recommendations

Issue #01 INFORMATIONAL

Gas observations

DESCRIPTION

1. **Recomputed scale factors**
   `_composeUsd` recomputes `10 ** feedDecimals`, `10 ** rateDecimals`, and `10 ** (rateDecimals + underlyingDecimals)` on every call though all are fixed at construction.
2. **Redundant SafeCast**
   `underlyingAnswer.toUint256()` runs its sign check right after the `underlyingAnswer <= 0` guard.

RECOMMENDATION

Precompute the scale factors as immutables and use `uint256(underlyingAnswer)` after the guard. Both are optional micro-optimizations.

RESOLUTION — ROUND 1 PARTIAL

The team fixed the scale factors. rateDecimals now lives in the base and the three 10 ** x values are computed once at construction as immutables. They kept the SafeCast after the sign guard on purpose, since it keeps forge lint clean without suppressions and stays a safety net if the guard is ever restructured.

3.2 StablePriceLib

`StablePriceLib` is a small stateless library holding the stable-price snap shared by the Aave v4 price feeds, mirroring the existing `PriceProviderV2` behavior. Its `snap` function forces a USD-stable price to exactly one dollar (`10 ** feedDecimals`) when the computed price lands within 1% of peg; a non-stable price, or a stable price outside the 1% band, passes through unchanged. It is invoked by `BaseAaveV4PriceFeed` after each price is composed.

Privileged Functions

None

Issues & Recommendations

No issues found.

3.3 ChainlinkPriceFeed

`ChainlinkPriceFeed` prices a single token for the Aave v4 oracle from a staleness-checked Chainlink aggregator. It supports two modes: with no underlying feed the aggregator is already USD-quoted (e.g. ETH/USD) and is only scaled to feed decimals, and with an underlying feed the aggregator is a rate in an underlying asset (e.g. weETH/ETH) whose price is multiplied by that underlying's USD price, read from any IAaveV4PriceFeed which enforces its own staleness.

It reads `latestRoundData`, requires a non-zero staleness bound at construction, and fails closed – `latestAnswer` reverts when the Chainlink feed is older than its bound, or when either leg is non-positive or reverts.

Privileged Functions

None

Issues & Recommendations

Issue #02 INFORMATIONAL

Redundant `SafeCast` after sign check

DESCRIPTION	`answer.toUint256()` runs its sign check immediately after the `answer <= 0` guard, so the cast's check is redundant.
RECOMMENDATION	Use `uint256(answer)` after the guard. Minor — the cast is cheap defensive code.

RESOLUTION — ROUND 1 ACKNOWLEDGED

3.4 PythPriceFeed

`PythPriceFeed` prices a token for the Aave v4 oracle from one of the MorphoPythOracle pair adapters deployed per pair on Optimism. Because the adapter enforces its own baked-in maximum age that is not the protocol's to control, this feed additionally discovers the Pyth feed ids the adapter prices with, read from the adapter at construction, and enforces its own `maxStaleness` bound directly against each Pyth publish time, so the effective freshness bound belongs to the protocol.

It supports the same two modes as the sibling feeds (a USD-quoted pair scaled to feed decimals, or a pair rate composed with an underlying's USD price) and fails closed, reverting on a stale Pyth publish time or a zero/reverting adapter price.

Privileged Functions

None

Issues & Recommendations

Issue #03 INFORMATIONAL

Naming and code-quality observations

DESCRIPTION

1. **Numbered feed ids** — `feedId1` – `feedId4` drop the adapter's base/quote meaning (`BASE_FEED_1/2`, `QUOTE_FEED_1/2`).
2. **Decimals naming** — the source decimals are named `oracleDecimals` here but `rateDecimals` in the three sibling feeds.
3. **Revert style** — `latestAnswer` / `_requireFresh` use `require(cond, Error())` while the sibling feeds and the base use `if (!cond) revert Error()`.
4. **Braceless `if`** — a condition and its `require` share one unbraced line.

RECOMMENDATION

Rename to `baseFeed1/2`, `quoteFeed1/2`, align `oracleDecimals` with `rateDecimals`, adopt the `if (!cond) revert Error()` style, and brace the combined `if`.

RESOLUTION — ROUND 1 RESOLVED

3.5 VedaAccountantPriceFeed

`VedaAccountantPriceFeed` prices a Veda `BoringVault` receipt token for the Aave v4 oracle from the vault's Accountant exchange rate. It reads the accountant's last-update timestamp and rejects a rate older than its staleness bound, then reads the exchange rate through `getRateSafe`, which reverts if the accountant has paused itself, rejects a zero rate, and composes the rate with the underlying asset's USD price when configured.

It requires a non-zero staleness bound at construction and fails closed, reverting on a paused or stale accountant, a non-positive or reverting underlying, or a zero rate.

Privileged Functions

None

Issues & Recommendations

Issue #04 INFORMATIONAL

Redundant second accountant call

DESCRIPTION	`latestAnswer` decodes the full `AccountantState` for only `lastUpdateTimestamp`, then calls `getRateSafe()` separately — though the struct already exposes `exchangeRate` and `isPaused`.
RECOMMENDATION	Derive the rate and pause state from the already-fetched `AccountantState` to save one external call.

RESOLUTION — ROUND 1 RESOLVED

3.6 OracleSinkPriceFeed

`OracleSinkPriceFeed` prices a token for the Aave v4 oracle from the `OracleSink`, the destination-chain store of prices relayed from the mainnet `PriceProvider` over LayerZero. The sink applies no staleness bound of its own, so this feed enforces an immutable `rateMaxStaleness` against the relay's stamped source-chain read time – the moment the relay read the price on mainnet, not the LayerZero delivery time – so a delayed delivery cannot present a stale price as fresh; the priced token is baked in because the sink is token-keyed.

It supports the same two-mode composition as the sibling feeds and fails closed, reverting when the relayed price is stale or non-positive, or for a token the sink has never priced.

Privileged Functions

None

Issues & Recommendations

Issue #05 INFORMATIONAL

Redundant `SafeCast` after sign check

DESCRIPTION	`answer.toUint256()` runs its sign check immediately after the `answer <= 0` guard, so the cast's check is redundant.
RECOMMENDATION	Use `uint256(answer)` after the guard. Minor — the cast is cheap defensive code.

RESOLUTION — ROUND 1 ACKNOWLEDGED

3.7 IAaveV4PriceFeed

IAaveV4PriceFeed is the minimal price-feed interface a price source implements to be read by the Aave v4 AaveOracle, declaring `decimals`, `description`, and `latestAnswer`. It is an MIT mirror of aave-v4's `IPriceFeed` ABI, declaring only the methods the protocol calls rather than vendoring the BUSL-licensed source.

The base contract and every concrete receipt-token feed implement it so those tokens can serve as Aave collateral, and composite feeds consume it as their underlying USD leg.

Privileged Functions

None

Issues & Recommendations

No issues found.

3.8 IVedaAccountant

`IVedaAccountant` is the slice of Veda's `AccountantWithRateProviders` that `VedaAccountantPriceFeed` reads. It declares `getRateSafe`, which returns the vault exchange rate in the base asset and reverts when the accountant is paused; `accountantState`, which returns the full accountant state struct including the last exchange-rate update timestamp; and `decimals`, the decimals of the exchange rate. It is an MIT mirror declaring only the methods the feed calls.

Privileged Functions

None

Issues & Recommendations

No issues found.

3.9 IOracleSink

`IOracleSink` is the slice of the cash-mainnet-asset-listing `OracleSink` that `OracleSinkPriceFeed` reads. It declares a token-keyed, Chainlink-shaped `latestRoundData(token)` accessor whose `updatedAt` is the source-chain time the relay read the price (not the LayerZero delivery time), and `decimals`, the sink's fixed price decimals. The sink applies no staleness bound of its own, leaving freshness enforcement to the consuming feed.

Privileged Functions

None

Issues & Recommendations

No issues found.


PALADIN
BLOCKCHAIN SECURITY